

SHL Conversational Assessment Recommender

This service is a stateless FastAPI API that uses an LLM-assisted two-pass pipeline for assessment recommendation. The request flow is: guardrails first, requirement extraction second, retrieval third, and response generation last. The system returns clarifying questions, relevant recommendations, refined results when the user changes constraints, and grounded comparisons that cite catalog evidence.

Design choices. I used a two-pass design to reduce hallucinations and keep the system explainable. Pass 1 extracts structured requirements from the conversation. Pass 2 uses retrieved catalog evidence to generate the final reply. I kept the API stateless so it is easy to deploy and easy to evaluate from full conversation transcripts. I added guardrails for prompt injection, off-topic requests, and turn limits before the LLM is called.

Retrieval setup. The default deployment path uses a lightweight hashing retriever so the service can run within Render free-tier memory limits. The code still supports an optional sentence-transformers / FAISS backend for local experimentation, but it is opt-in. Retrieved items are filtered by job level and test type when possible, then validated against the catalog before returning URLs.

Prompt design. The system prompt constrains scope to SHL assessments and prevents invented URLs. The extraction prompt returns JSON fields such as role, seniority, skills, test preferences, off-topic flags, and comparison intent. The recommendation and comparison prompts receive only catalog-grounded context plus the user conversation, which keeps the output focused and traceable.

Evaluation. I used schema tests, guardrail tests, and conversation-oriented checks to validate behavior. The test suite covers response schema correctness, injection detection, off-topic refusal, turn counting, and catalog validation. I also used replay-style manual evaluation on sample conversations to check whether the system asked clarifying questions when needed and returned grounded recommendations when enough context was available.

What did not work. A single-pass generation approach caused hallucinated assessment names. Pure keyword matching missed semantic intent. Eagerly loading sentence-transformers and FAISS on startup was too heavy for a small Render instance and caused memory issues. I replaced those defaults with a lighter backend and made the heavier stack optional.

How improvement was measured. Improvement was measured by whether the service could answer the evaluator conversations without schema failures, whether recommendations remained inside the catalog, whether off-topic and injection attempts were refused, and whether the app could start reliably in the target deployment environment. I verified this with automated tests plus runtime startup checks.

Implementation and Deployment Notes

The app exposes GET /health and POST /chat. The deployment config is set up for Render using Docker, and the app now binds successfully under a small-memory environment by avoiding heavy model downloads during startup. The lightweight retrieval backend is the production default; the optional semantic backend can be enabled later if the deployment target has more memory.

AI tools used. I used Google Gemini to help implement and iterate on the solution, along with Copilot-assisted editing and validation in the workspace. All changes were reviewed and tested locally before finalizing the deployment path.

Final validation. The repository test suite passes, and the Uvicorn server starts successfully in the current workspace using the lightweight retrieval mode.